

C语言与程序设计

The C Programming Language

第5章 函数

王多强

1097412466

dqwang@hust.edu.cn

华中科技大学计算机学院



本章内容

5.1 结构化程序设计和C程序的一般结构

5.2 函数的定义、声明和调用

5.3 变量的存储类型

5.4 作用域与可见性

5.5 存储类型

5.1 结构化程序设计和C程序的一般结构

1. 结构化程序设计

结构化编程是构造程序的一种策略，它包括2条编程标准：

(1) **程序的控制流应该尽可能简单**：由三种基本结构构成：
顺序、分支、循环。

(2) **模块化设计**：采用**自顶向下、逐步求精**的方式。

即，**把大问题分成若干个子问题**（或**把大程序分解为若干个子程序**）。如果一次分解后子程序还比较大，则可继续将子程序分解成更小的子程序，**直到可直接实现**。分解后得到的每个子程序称为一个**模块**。然后逐个模块编程实现。而**整个程序的功能通过子程序间的协作（调用）实现**。

例 设计一个程序，计算n!。

实现1：整个程序只有一个main函数，**程序代码都写在main函数**
中，

```
int main(void)
{
    int n=4;
    long f=1;
    scanf("%d", &n);
    for (; n>0; --n) f *= n;
    printf("%ld\n", f);
    return 0;
}
```

存在的问题：main函数语句多，内容复杂，结构不够清晰；
无法合作开发；**代码复用差。**

实现2：分成两个模块/函数设计。其中一个模块（函数**fac**）专门负责计算参数n的阶乘，另一个模块（main函数）是总控过程，负责输入n，然后调用计算模块实施计算，最后实现整个程序的功能要求。

```
long fac(int n)
{
    long f=1;
    for (; n>0; --n) f *= n;
    return f;
}
```

```
int main(void)
{
    int n=4;
    scanf("%d", &n);
    printf("%ld\n", fac(n));
    return 0;
}
```

好处：**程序模块化**，结构清晰，每个函数具有不同的功能、做不同的事，函数语句结构和功能相对简单，易于实现；便于多人协作开发；**代码复用性强**。

2. C程序的基本结构

C程序采用**函数**的形式具体实现模块化编程

- ◆ 一个C程序由一个或多个函数组成（其中有且只有一个main函数）。**函数是C程序的基本组成单位**（对应模块）。
 - ◆ 程序员逐一编写函数实现各模块的功能。
 - ◆ 程序的执行从main函数开始，通过函数间的调用实现整个程序的功能（**阅读教材5.1节**）。
- C程序中的函数分两大类：
- ◆ 由系统提供的**标准函数**（如 printf, getchar等）；
 - ◆ 程序员自己编写的**用户自定义函数**。
- 本章主要讲如何编写自定义函数。**

结构化程序设计的好处

- 程序逻辑清晰，编制方便，易于管理、修改和调试。
- 增强了程序的可读性、可维护性和可扩充性，方便多人分工合作完成程序的编制。
- **函数可以公用**，避免在程序中编写重复的代码。
 - 公用代码称为**代码重用**（**代码复用**）。

5.1.1节：水仙花数程序的设计（自学）

—— 进一步巩固对结构化程序设计、模块化程序设计的理解。

5.1.2 蒙特卡罗模拟：猜数游戏(自学)

学习要点：

- ◆ 什么是**蒙特卡洛法**？侧重于解决什么类型的问题。
- ◆ **随机数发生器函数**的使用：什么是**种子**？什么是**伪随机**？
- ◆ 获取**系统时间**的函数的使用。
- ◆ 学习并深入理解猜数游戏程序的构造思路 and 实现策略。

例5.1 编写一个猜数的游戏程序：计算机产生一个1到1000之间的**随机数**。把该数作为要猜的数，要玩游戏的人猜出该数。

游戏者输入所猜的数，如果猜得不正确，继续猜直到正确为止，同时计算游戏者猜数的次数。

为了帮助游戏者一步一步得到正确答案，程序会不断地发出 “Too high” 或 “Too low” 信息。最后，程序向游戏者显示游戏结果。

自顶向下的分析和分解问题：

- 既然是一个游戏程序，就应该允许玩家反复玩多次，直到不想玩为止，所以应该有一个循环结构控制玩游戏的流程。
- 玩一次游戏的大任务可以分解成以下两个子任务：
 - (1) 计算机产生一个1到1000的随机数供游戏者猜测；
 - (2) 游戏者猜数，直至猜对。

可得主程序结构：

```
do {  
    计算机产生一个1到1000的随机数  
    游戏者猜数，直至猜对  
    继续玩吗？不想玩则break;  
} while (继续);
```

自顶向下的分解：

子任务1：产生一个1到1000的随机数

步骤：

- ① 调用标准库函数rand产生一个随机数；
- ② 将这个随机数限制在1 ~ 1000之间。

描述：

把程序中特定的功能模块设计成**函数**，以供随时调用：

- 1) 简化主函数的流程，不要只有一个main函数；
- 2) 简单直观，提高程序的易读性和可维护性。

- 把任务1设计成一个独立的函数：**GetNum(void)**

```
/******
```

函数名称：GetNum

函数功能：产生一个1到MAX_NUMBER之间的随机数，供游戏者猜测。

函数参数：无

函数返回值：返回产生的随机数

```
*****/
```

```
#define MAX_NUMBER 1000
```

```
int GetNum(void)
```

```
{
```

```
    int x;
```

```
    printf("A magic number between 1 and %d has been chosen.\n", MAX_NUMBER);
```

```
    x=rand();
```

/* 调用标准库函数rand产生一个随机数 */

```
    x= x % MAX_NUMBER + 1; /* 将这个随机数限制在1 ~ MAX_NUMBER之间 */
```

```
    return(x);
```

/* 返回这个随机数给调用者， */

```
}
```

- rand是stdlib.h中的一个函数，它返回一个非负并且不大于常量RAND_MAX的随机整数，RAND_MAX的值取决于计算机系统。
- MAX_NUMBER是用#define定义的符号常量，其值为1000。

子任务2：游戏者猜数，直至猜对，流程描述如下

```
for(;;) {
```

```
    输入猜测的数
```

```
    if (猜对了) 结束
```

```
    else if (小了) 输出太小的提示
```

```
    else 输出太大的提示
```

```
}
```

■ 将任务2也设计成一个独立的函数： **GuessNum(int x)**

```
/******
```

函数名称：GuessNum

函数功能：游戏者根据提示反复猜数，直至猜对为止，输出猜数的次数。

函数参数：x表示被猜的数

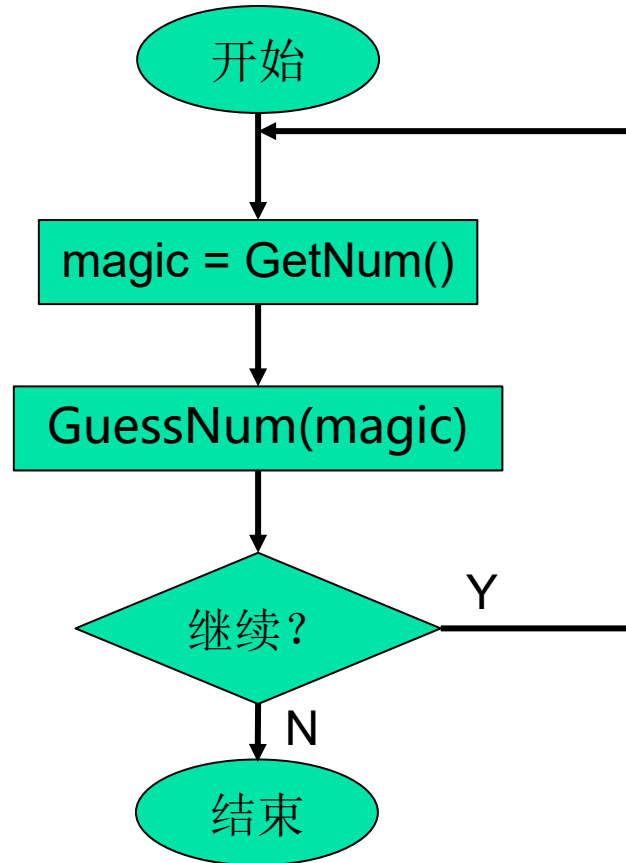
函数返回值：无

```
*****/
```

```
void GuessNum( int x )
{
    int guess, counter = 0;
    for ( ; ; ) {
        printf("guess it: ");
        scanf("%d", &guess);
        counter++;          /* 统计猜的次数 */
        if (guess == x) {    /* 猜对 */
            printf("You guessed the number by %d times!\n\n", counter);
            return;
        }
        else if (guess < x) /* 猜小了 */
            printf("Too low. Try again.\n");
        else /* 猜大了 */
            printf("Too high. Try again.\n");
    }
}
```

最后设计主控程序 —— 主函数main

- 任何C程序都必须**有且只有一个main函数**，是程序的入口点。
- main函数中将调用GetNum和GuessNum函数完成程序的功能。



```

#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#define MAX_NUMBER 1000
int GetNum (void);      /* 函数原型 */
void GuessNum(int) ;    /* 函数原型 */
int main(void)
{
    char command;
    int magic;
    printf("This is a guessing game\n\n");
    srand(time(NULL));   /* 用系统时间初始化随机数生成器 */
    do {
        magic = GetNum( );          /* 调用GetNum产生随机数供猜测 */
        GuessNum(magic);            /* 调用GuessNum猜出这个数 */
        printf("Play again? (Y/N) ");
        scanf("%1s", &command);    /* 询问是否继续 */
    } while (command == 'y' || command == 'Y');
    return 0;
}

```

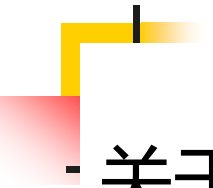
- 函数srand和rand的原型在stdlib.h中；函数time的原型在time.h中
- 调用函数srand初始化**随机数种子**，可以采用系统时间（由time函数得到）作为种子。



进一步说明：

- 对现实世界中的随机事件要用**模拟算法**去模拟。
 - 在程序设计中，可使用**随机数函数**来模拟现实中不可预测情况，称为**蒙特卡罗模拟**。
- 用**rand**函数生成随机数。 **rand**函数称为**随机数发生器**。
 - 该发生器从称为种子的初始值开始用“**确定的算法**”产生随机数。
 - **种子**：一个无符号整型数
 - 通过种子产生第一个随机数后，后续的随机序列也就是确定的了，这种**依靠计算机内部算法产生的“随机”数称为伪随机数**。
- **rand**产生的随机数依赖于种子。
 - 为了使程序在反复运行时能产生不同的随机数，必须改变这个种子的值，这称为**初始化随机数发生器**，由函数**srand**来实现。

5.2 C函数的定义、声明和调用



- 关于函数，涉及三个方面的内容：

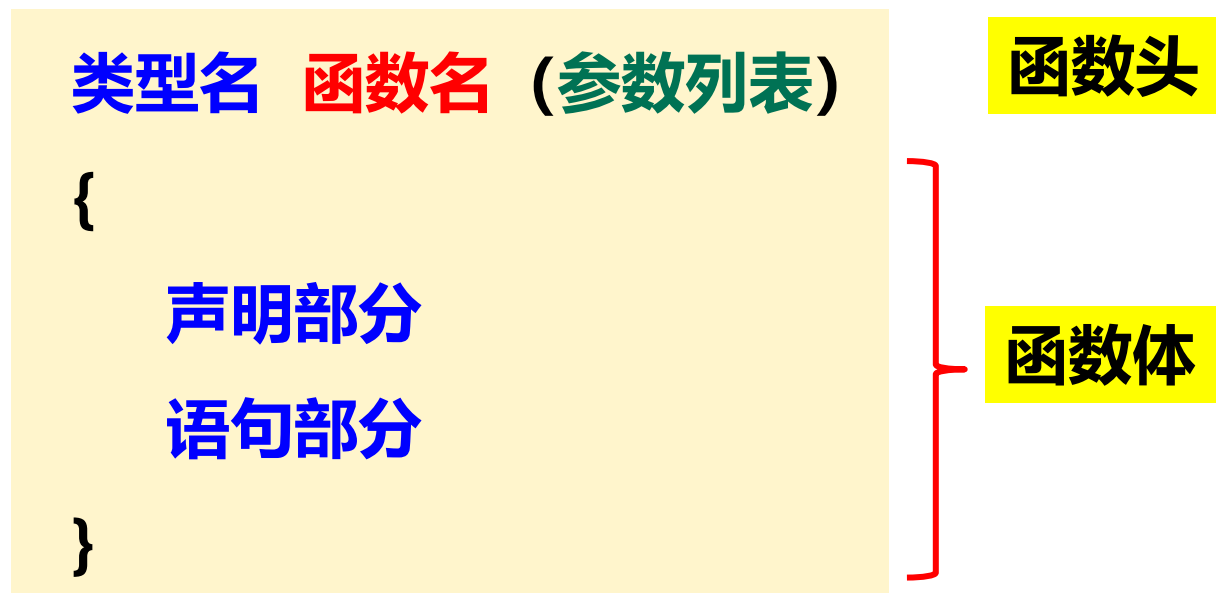
① 函数的定义

② 函数的声明

③ 函数的调用

5.2.1 函数的定义

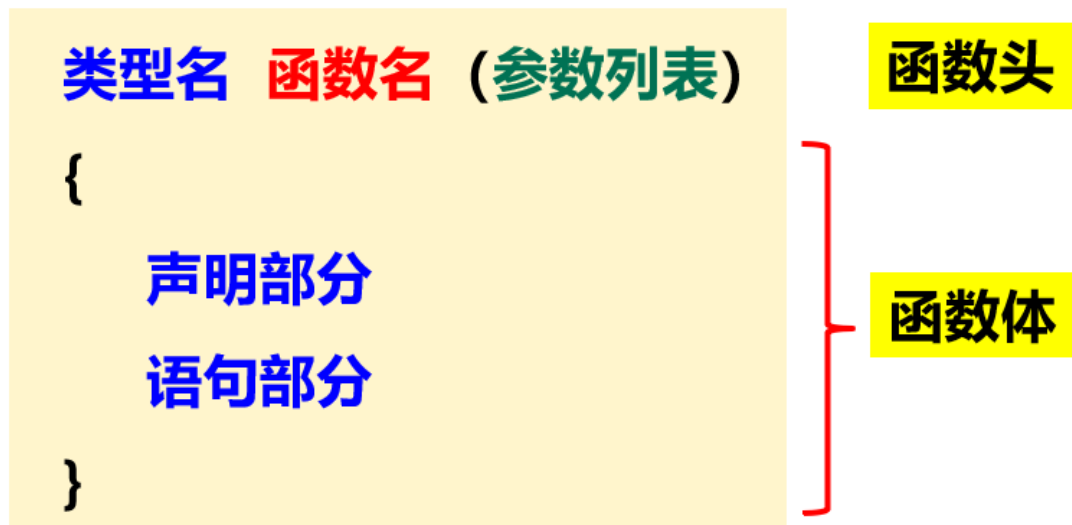
定义函数的一般形式为：



类型名： 指函数返回值的数据类型名。

函数返回值的数据类型简称为**函数的类型**。

- ◆ 函数的类型可以是**除数组以外**的任何类型，如int；
- ◆ 当函数没有返回值时，类型名要写为**void**。



参数列表：说明函数入口参数的**类型、名称、个数和顺序**，有多个时，参数之间用逗号分隔。

- ◆ 必须**明确列出每一个参数的类型和名称**，**不能"合写"**。

如：int max(int x, int y) ✓

int max(int x, y) ✗

- ◆ 如果函数没有参数，参数列表可为空或说明为**void**。

函数的返回值

函数的计算结果称为**函数的返回值**。

有返回值的函数要通过**return语句**返回返回值。

- return语句有两种基本形式：

(1) **return;** /*不带表达式，仅适用于void函数*/

(2) **return 表达式;** 或 **return (表达式);**

非void函数必须要用return返回返回值。return后面**表达式**的值即为函数的返回值，其类型要与函数定义时声明的函数类型**一致或兼容**——类型不一致时，能会把表达式值自动转换为函数类型要求的值；如果不能实现自动转换则编译器会报错（类型不匹配）或返回意想不到的结果。

例5.3 写一个函数IsPrime，判断整数参数n是否为素数。如果n是素数，则返回1；否则返回0。

```
int IsPrime(int n)
{
    int k, limit;
    if (n == 2) return 1;
    if (!(n%2)) return 0;
    limit = n/2;
    for (k=3; k<=limit; k+=2)
        if (!(n%k)) return 0;
    return 1;
}
```

- ◆ 函数内可以有**多个return语句**，每个return语句都是程序根据处理逻辑设置的一个返回点。
- ◆ 程序执行时，会根据实际情况执行到其中之一，而一旦执行到某个return语句，则**立即结束整个函数的执行并返回**，函数中的其它语句将不再执行。

■ 函数的返回

一个**被调函数**执行结束并返回后，**程序的控制流**立刻转移到

主调函数中的**调用点**并继续执行调用点后面的其它操作和语句。

(如果该函数是**main函数**，则结束整个程序的执行并返回到**操作系统**)

```
int IsPrime(int n)
{
    int k,limit;
    if (n == 2) return 1;
    if (!(n%2)) return 0;
    limit = n/2;
    for (k=3; k<=limit; k+=2)
        if (!(n%k)) return 0;
    return 1;
}
```

```
int main()
{
    int x;
    printf("input a integer:");
    scanf("%d",&x);

    if(IsPrime(x))
        printf("%d is a prime.\n",x);
    else printf("%d is not a prime.\n",x);
    return 0;
}
```

在main函数中调用IsPrime函数
先调用IsPrime函数，返回时再
进行if条件判断

- ◆ **void类型的函数可以没有return语句**，此时程序执行到函数体的**右花括号**处自动结束。如：

```
void show(char str[ ])
```

```
{
```

```
    printf("This is the first program for %s to "  
        "learn C programming!\n",str);
```

这里用到字符串的拼接

```
}
```

而如果有return语句，return后面不能跟任何表达式。

- 对被调函数的返回值，主调函数中可以使用，也可以不使用。

如：char **c=getchar();** 取得函数返回值，赋给c变量

或： **getchar();** 只调用函数，不使用其返回值

- **内联函数**：**inline**，C99增加的关键字。

内联函数声明：**在函数类型名前面加关键字inline**

如：**inline** int add(int a, int b)
{
 return a+b;
}

编译时，编译器将程序中出现的内联函数的**调用表达式用内联函数的函数体来进行替换**。

- **好处**：使用内联函数不会产生因函数调用而带来的开销，执行效率高；
- **坏处**：原来的函数调用形式被替换成函数体代码，编译后会增加可执行程序的长度（代码量增加），进而增加空间开销。

内联函数是以增加目标代码量为代价来换取执行时间上的节省。

inline函数仅仅是编译建议，最后能否真正内联，要看编译器的意思，它如果认为函数不复杂，能在调用点展开，就会真正内联。

■ 一些具体要求

- ◆ 内联函数要**比较简洁**，只能有少量语句。如果语句较多，则不适合定义为内联函数，也不能真正实现内联。
- ◆ 内联函数体中**不能有循环语句、if语句或switch语句**，否则，即使函数定义时使用了inline关键字，编译器也会把该函数当作普通函数处理。
- ◆ 因为内联函数要在调用点展开，所以编译器必须随处可见内联函数的定义，这要求**内联函数必须在调用之前定义**，并且在每个调用了内联函数的文件中都要有该内联函数的定义（**inline函数可以被重复定义**。并通常将内联函数的定义放在头文件中共享）。

5.2.2 函数原型

- ◆ C语言要求函数必须**先声明后使用**。

函数的声明：就是告知编译器该函数的存在，说明该函数的名字、类型、参数等情况，以便于编译器在遇到对该函数的调用时能够“**认得它**”，从而可以正确编译，否则可能会出问题。

- ◆ 函数声明有两种方式：

(1) 直接通过函数定义声明

此时，要**把函数的定义写在函数的调用之前**的位置。

如: `int GetNum(void)`
`{`
`.....`
`}`
`void GuessNum(int x )`
`{`
`.....`
`}`
`int main(void)`
`{`
`.....`
`magic = GetNum( );`
`GuessNum(magic);`
`.....`
`}`

函数定义在前

函数调用在后

先定义函数再调用它的方式叫**先定义后使用**。

(2) 通过函数原型声明

如果函数的定义出现在函数的调用之后（或被调用函数在其它文件中定义），则必须在函数调用之前给出**函数原型声明**。否则编译时编译器可能无法正确辨认该函数而出错。

■ 函数原型：

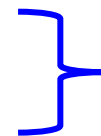
类型名 函数名(参数类型表);

如： `int GetNum(void) ;`
`void GuessNum(int x );`

函数原型就是函数头后面加分号后得到的一行语句。

如：

```
int GetNum(void) ;  
void GuessNum(int x );  
int main(void)  
{  
    ...  
    magic = GetNum( );  
    GuessNum(magic);  
    ...  
}  
int GetNum(void)  
{  
    ...  
}  
void GuessNum(int x )  
{  
    ...  
}
```



函数原型声明



函数调用在前



函数定义在后

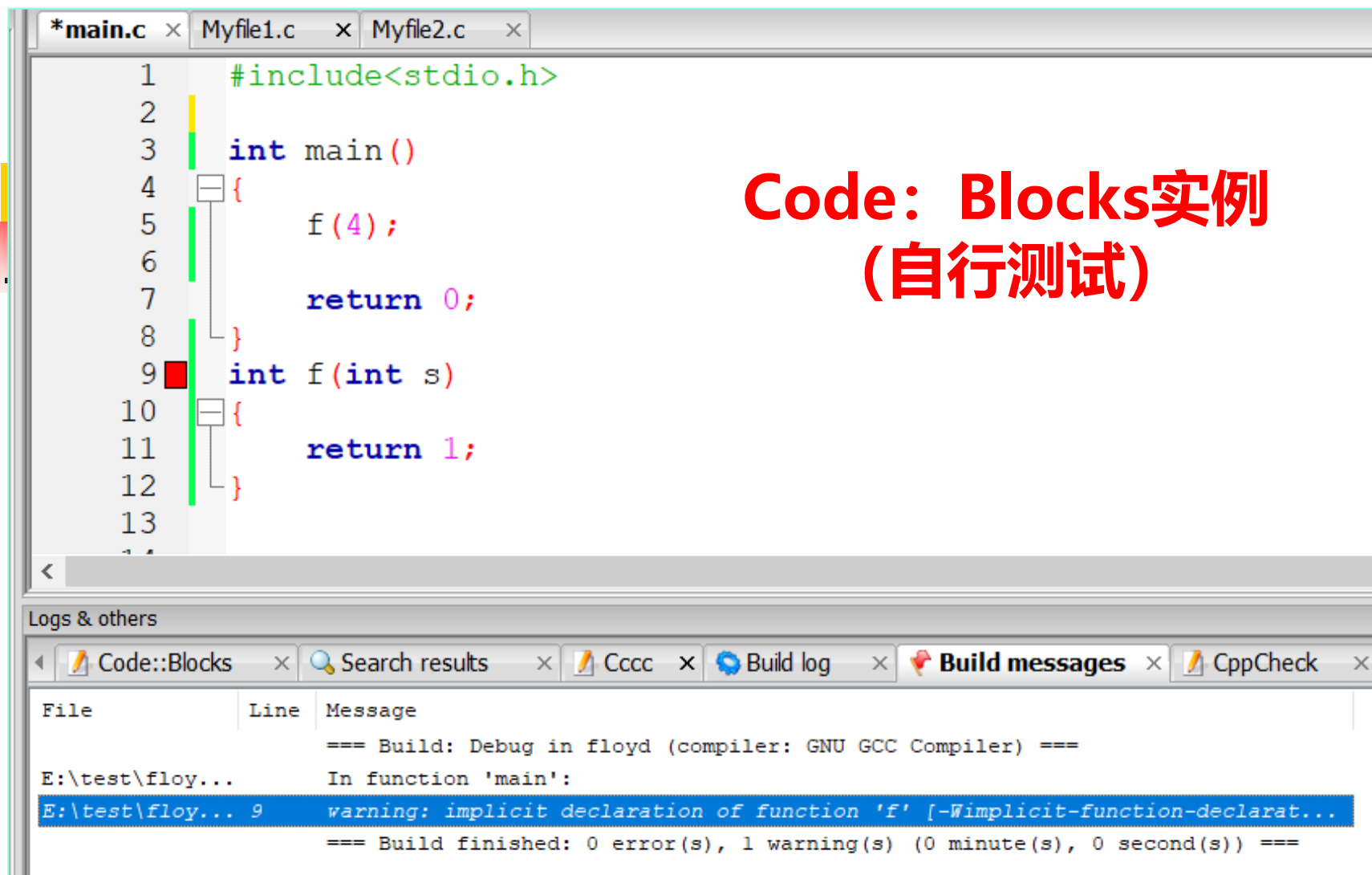
如果函数定义在后又遗漏了函数原型声明会有什么后果？

编译器在首次遇到没有声明而调用的函数时，将为其构造一个形如：**int f();** 的**缺省声明**并继续编译 (函数**默认类型是int**，且**没有参数表**信息)。如果在后面实际碰到了该函数的定义或声明，则可能：

- ◆ 如果该函数实际上也是int类型且没有参数，则不会出问题。
- ◆ 如果**函数类型不是int**，则给出一个错误提示并终止编译：

Type mismatch in redeclaration of 'f'

- ◆ 如果**函数类型是int**，但**参数表不空**，此时不提示错误 (也可能是警告信息)，而且编译器对参数不作检查，继续编译生成可执行代码。但执行时，可能会**因参数不匹配而产生意想不到的错误**。（C语言认为把正确类型的参数传递给函数是程序员的责任）



The screenshot shows the Code::Blocks IDE with a C program open in the editor. The program defines a function `f` and calls it from `main`. The build output window at the bottom shows a warning about the implicit declaration of function `f`, but no errors.

```
1  #include<stdio.h>
2
3  int main()
4  {
5      f(4);
6
7      return 0;
8  }
9  int f(int s)
10 {
11     return 1;
12 }
13
```

**Code: Blocks实例
(自行测试)**

Logs & others

- Code::Blocks
- Search results
- Cccc
- Build log
- Build messages
- CppCheck

File	Line	Message
=== Build: Debug in floyd (compiler: GNU GCC Compiler) ===		
E:\test\floy...		In function 'main':
E:\test\floy... 9		warning: implicit declaration of function 'f' [-Wimplicit-function-declarat...]
=== Build finished: 0 error(s), 1 warning(s) (0 minute(s), 0 second(s)) ===		

这里，事先没有给出函数的声明，CB编译时有一个警告信息，但对参数的一致没有提示错误。

- ◆ 程序员应养成良好的编程习惯：

在函数调用之前必须给出函数的定义或原型声明。

- ◆ 良好的编程风格还有：

- 在每个函数的上方写上**函数注释**：说明函数的功能、作者、时间和其他相关信息。
- 为函数命名时**选择有意义的名称**。
- **Windows风格函数名**：函数名用“**动词**”或者“**动词+名词**”的**动宾词组**形式命名，且每个逻辑部分的首字母大写。

如：**int GetNum(void)**

这样可以帮你减少很多不必要的错误且易于程序后期维护

- ◆ **函数原型参数表中的形参名可以省略，但类型名不能少，且顺序、个数必须和函数定义一致。**

例如， `void GuessNum( int );`

- **也可以写与函数定义中的形参名不一样的形参名。**

例如， `void GuessNum( int a );`

函数原型相对函数定义是独立存在的。 编译器也只关心函数原型中参数的**类型、个数和顺序**，形参名不是函数原型必需的（函数原型声明中没有函数体，不涉及对形参的使用）。所以函数原型中参数的名字可以和函数定义不同甚至省略。

5.2.3 函数调用

1. 函数调用表达式的一般形式:

函数名(**实参列表**)

如果是无参函数，则实参列表为空，即：**函数名()**

括号里不能写void

函数调用可以出现在以下位置:

(1) 作为**独立表达式语句**出现。如:

```
putchar(c);
```

(2) 作为另一个表达式中的一个**“操作数”**出现。如:

```
c=getchar();
```

(3) 作为另一个函数调用的一个**“实参”**出现 (**嵌套调用**)。

如: `printf("%10.0f", sqrt(x));`

2. 函数调用的执行过程

例5.4 设计一个函数，计算n!，并调用。

```
#include<stdio.h>
```

```
long fac(int); /* 函数原型 */
```

```
int main(void)
```

```
{
```

```
    int n=4;
```

```
    printf("%d\n", n);
```

```
    printf("%ld\n", fac(n));
```

```
    printf("%d\n", n);
```

```
    return 0;
```

```
}
```

```
long fac(int n)
```

```
{
```

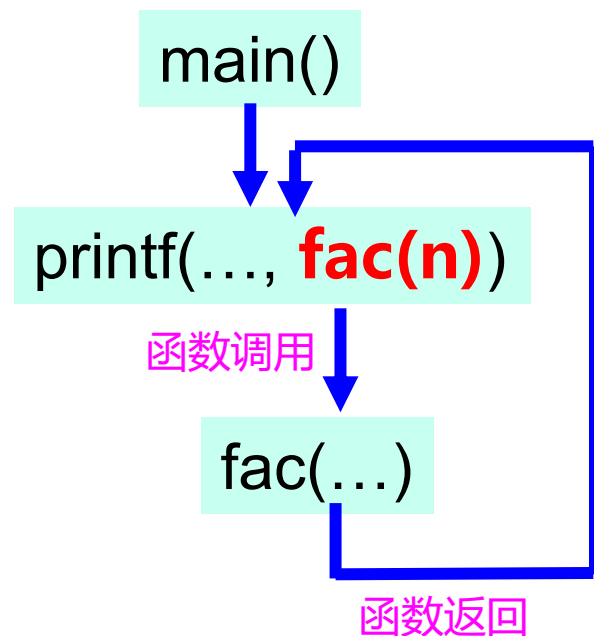
```
    long f=1;
```

```
    for (; n>0; --n) f *= n;
```

```
    printf("%d\n", n);
```

```
    return f;
```

```
}
```



程序的输出结果:

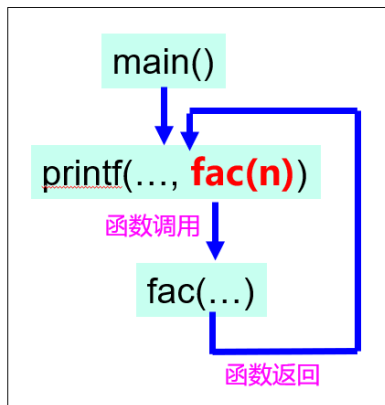
4

0

24

4

形参是实参的副本，
对形参的任何改变都
不影响实参。



```
long fac(int n)
{
    long f=1;
    for (; n>0; --n) f *= n;
    printf("%d\n", n);
    return f;
}
```

对一次函数调用的简单描述：

当main函数中发生对fac函数的调用时，

- ◆ 首先main函数将其实参——当前n的值传递给fac函数的形参n；然后转而执行fac函数，计算得到n!的值f。
- ◆ 完成计算后，fac函数的返回语句return f; 结束fac函数的执行，然后返回到main函数中的调用点。
- ◆ main函数获得此次调用fac函数的返回值，之后按照指定的方式输出——这样对fac函数的一次调用就完成了。

函数调用的规范：包括参数传递、函数执行、函数返回

(1) 参数传递

函数调用时将实参传递给形参的行为称为**参数传递**。C语言的参数传递方式是**值传递**。

- **值传递**：是指参数传递时，仅将**实参的值**单向复制给相应的**形参**：① 形参是实参的“**副本**”，独立存在；
② 对形参值的修改不影响实参。

```
int main(void)
{
    int n=4;
    printf("%d\n", n);
    printf("%ld\n", fac(n));
    printf("%d\n", n);
    return 0;
}
```

```
long fac(int n)
{
    long f=1;
    for (; n>0; --n) f *= n;
    printf("%d\n", n);
    return f;
}
```

即使是实参和形参同名，但二者除了调用时有值的复制，其它没有任何直接关联

实参单元

$n=4$



函数调用时将**实参的值**复制给形参

形参单元

$n=4$



$n--$

$n==3$



$n--$

$n==2$

...



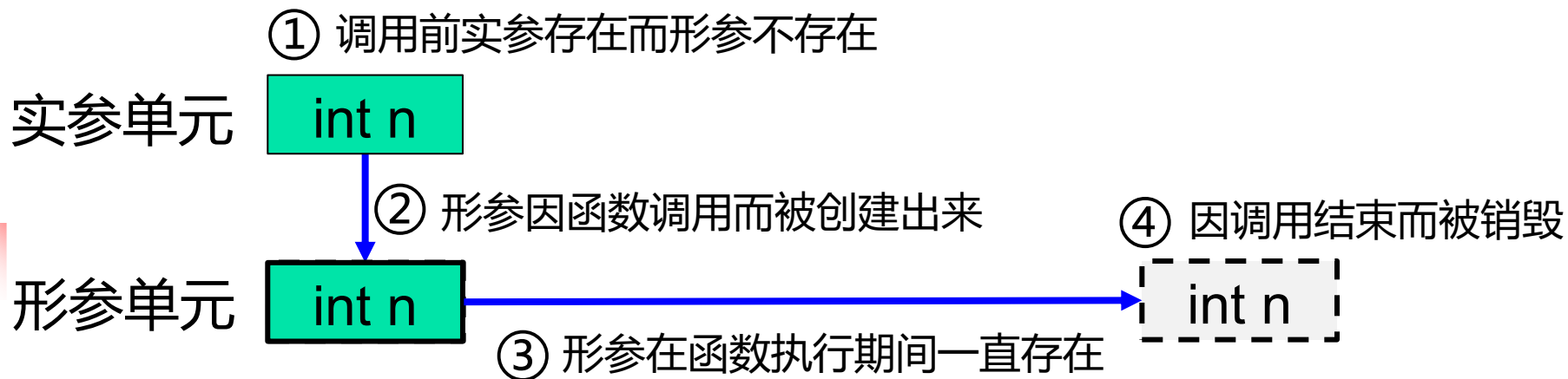
$n--$

$n==0$

被调函数中的计算

即使同名，实参和形参也没有关联

实参和形参相互独立，对形参的改变不影响实参



形参的产生、使用和消亡：

- ① 调用前实参存在；但形参是不存在的。
- ② 根据C语言的函数调用规则，形参只有在调用的那一刻因为系统为其分配存储单元而产生。
- ③ 在整个函数执行期间，形参是存在的；
- ④ 但函数执行完返回时，形参单元会被系统释放，形参就此也就不存在了。下一次函数若再被调用，重复该“**创建—使用—销毁**”的过程。所以形参的生存周期是短暂的，其“创建—使用—销毁”在每一次函数调用时都会发生。

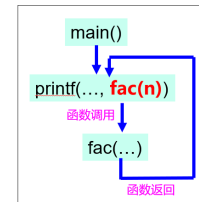
根据参数类型的不同，C语言的值传递又分为**普通的值传递**和**地址传递**。

址传递：参数传递的值是一个变量的地址。

此时，形参将是**地址型参数（指针参数）**，而传递给形参的实参也必须是**地址型的量**。如：标准库函数scanf

```
scanf("%d", &x);
```

- 什么是地址型变量，如何定义地址型变量将在第8章中介绍。
- **地址传递本质上是值传递的一种**，只是传递的参数值是地址型的数据。除此之外，其他性质与前面讲的是一样的。

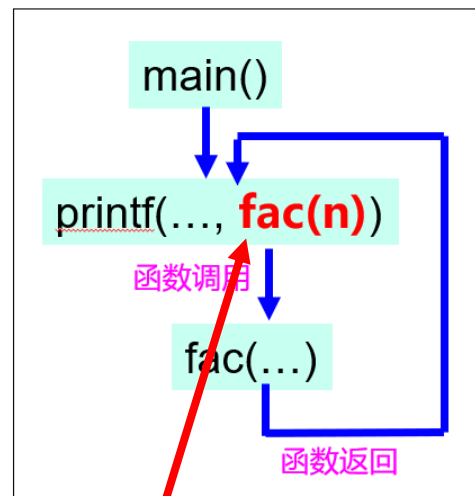


(2) 被调函数的执行:

参数由主调函数传递给被调函数后，就**暂停**主调函数的执行而转向被调函数执行，之后：

- ◆ 从被调函数的第一条可执行语句开始依次执行，直至完成被调函数的所有操作（这个过程和main函数的执行是一样的）。如果在被调函数中又存在对其它函数的调用，那么将按照同样的规则处理下一个层次的新函数调用。
- ◆ 当被调函数执行完所有可能的操作后，即刻终止被调函数的执行，然后进行返回。如果有返回值，此时将准备好返回值。

(3) 函数返回



在程序的控制流从主调函数转向被调函数的那一刻，主调函数有一个因转向被调函数执行而暂停自己执行的**暂停点**。

而从被调函数返回主调函数的这一刻，程序控制流将回到主调函数，并从刚才的**暂停点**处继续执行主调函数中被“中止”的运算。如果被调函数有返回值，这一刻主调函数也将获得该返回值，并将其引用到主调函数的后续运算中。

5.3 变量的存储类型

C语言中，变量有两种基本属性：

- ◆ **数据类型**：决定变量存储空间的大小、数据的取值范围、对数据的操作和运算；
- ◆ **存储类型**：决定变量的**作用域、存储方式、生存期、初始化方式**。

1、作用域

作用域指**标识符的有效范围**。标识符包括**变量名、函数名**等。

标准C定义了4种类型的作用域范围：

- (1) **文件作用域**：开始于文件开头，结束于文件结尾（.c 或 .h等文件）。在文件范围说明的标识符在整个文件内都是有效的，通常是全局变量名、函数名等。
- (2) **块作用域**：指程序块（即复合语句）范围，开始于左花括号“{”，结束于右花括号“}”。在块范围内说明的标识符只在本块内有效，出了其所在的程序块即失效，通常是局部变量名等。

(3) **函数原型作用域**：开始于**函数原型**的左括号 “(” ，结束于函数原型的右括号 “)” 。在函数原型中说明的标识符（即形参名）只在函数原型内有效，出了函数原型即失效。

如： `double power(int x, int n);`

(4) **函数作用域**：开始于函数体的左大花括号 “{” ，结束于函数体的右大花括号 “}” 。在函数范围内说明的标识符只在函数体内部有效，出了函数即失效，包括函数内定义的局部变量名、标号，以及参数表里的形参名。

2、 局部变量和全局变量

全局变量：在文件范围内定义、在所有函数（包括main函数）之外的变量称为**全局变量**，也称为**外部变量**。

- ◆ **全局变量具有文件作用域**：其有效范围从其定义处开始一直到其所在文件的末尾；并且可以通过**外部声明**将其作用域扩展至程序的其他模块和定义之前的区域。
- ◆ 全局变量可被其作用域范围内的所有函数共享使用。

```
int glb_x;  
int main(void)  
{  
    glb_x=5;  
    ...  
}  
void show()  
{  
    printf("glb_x=%d", glb_x)  
}
```

- ◆ **全局变量在整个程序内不可重名**。静态全局变量除外，见后。

局部变量：在程序块（包括函数体、复合语句等）内部定义的变量称为**局部变量**。

- ◆ **局部变量只有局部作用域（块作用域或函数作用域）**：其有效范围从其定义之处开始到其所在的程序块结束位置。
- ◆ 不同程序块中的局部变量是相互独立的，有各自独立的存储空间。
- ◆ **同一作用域内的局部变量不能重名，但不同作用域的局部变量可以重名，互不影响。**

```
int main(void)
{
    int x;
    int y;
    x=5;
    y=power2(x);
    ...
}
int power2(int y)
{
    int x;
    x=y*y;
    return x;
}
```

形参视为局部变量的一种

全局变量的使用：

全局变量有文件作用域，可被其作用域范围内的所有函数共享使用，那么在一个函数里为全局变量赋的值可以在另一个函数里使用。

(1) **全局变量可以起到“带入”、“带出”数据的作用**，使得函数有更多的渠道可以“**带入参数**”或“**返回结果**”，而不局限于参数表或return语句一次只能返回一个结果。当必须在不同函数之间**共享变量**时，全局变量是很有用的。

```
int x;  
int y;  
void x_power();  
int main(void)  
{  
    x=5;  
    x_power();  
    printf("%d", y);  
    ...  
}  
void x_power()  
{  
    y=x*x;  
}
```

(2) 但使用全局变量在函数间传递数据，会使得函数之间的**耦合度**加大，有一定的副作用，**要慎用**。

使用全局变量交换数据会降低函数的独立性，并很难在其他程序中复用依赖于全局变量的函数。

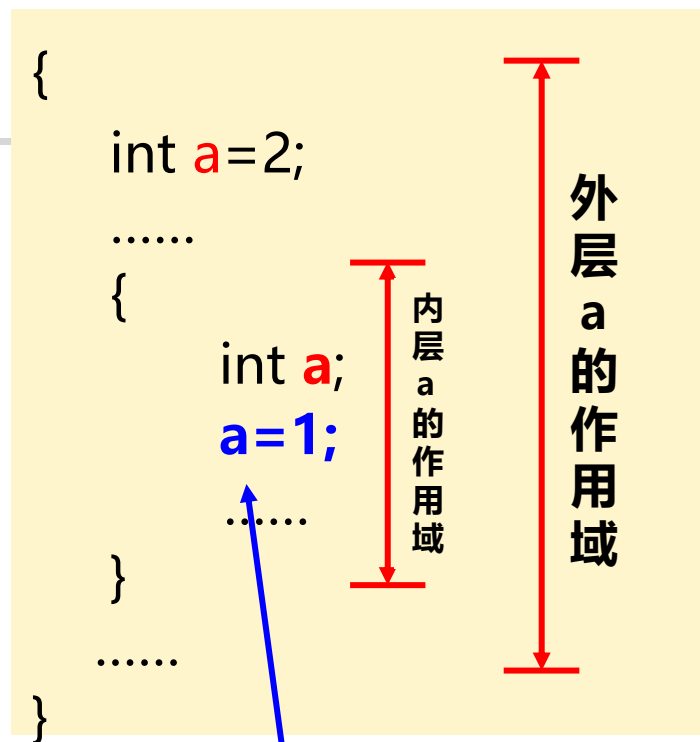
(试想，在程序维护期间，如果改变全局变量，比如改变全局变量的类型或名称，那么将需要检查程序中的每个使用该变量的函数，以确认该变化对函数产生的影响；或者全局变量被错误赋了值，也可能很难确定出错的函数)。

因此从结构化程序设计的观点来看，**不要过于依赖全局变量，少用全局变量**。对函数而言，在大多数情况下通过形式参数和返回值（return语句）进行通信比通过全局变量通信的方法更好、更安全。这样有助于提高函数的局部性，降低副作用。

3、可见性

◆ 作用域的嵌套：

程序块可以嵌套，形成嵌套的作用域：每个块都可以定义自己的变量，而且不同块内的变量可以重名。如果一个变量名a同时出现在外层块和内层块中，那么外层a的作用域包含了内层a的作用域，这称为作用域的嵌套。



问题：如果在内、外层作用域内定义了同名变量，如a，那么在内层使用的a是内层定义的变量a还是外层定义的变量a？

C语言规定：当内层定义了和外层同名的变量时，在内层里，外层的变量暂时**失去可见性，即不能用**。

◆ **可见性**：指一个标识符是否实际可用。

外层定义的变量在内层范围内是有效的，但未必可用：

```
{  
    int a=2,b=4;  
    printf("a=%d, b=%d\n", a, b);    /*输出 a=2, b=4 */  
    {  
        int a;  
        a=3; b=5;    /*a是内层的a, b是外层的b*/  
        printf("a=%d,b=%d\n", a, b); /*输出 a=3, b=5 */  
    }  
    printf("a=%d,b=%d\n", a, b);    /*输出 a=2, b=5 */  
}
```

内层
a的
作用域

外层
a的
作用域

- 对变量a，由于内层作用域有同名变量，所以外层的a在内层不可见。
- 对变量b，在内层没有同名定义，它在内、外层程序块中都可见。

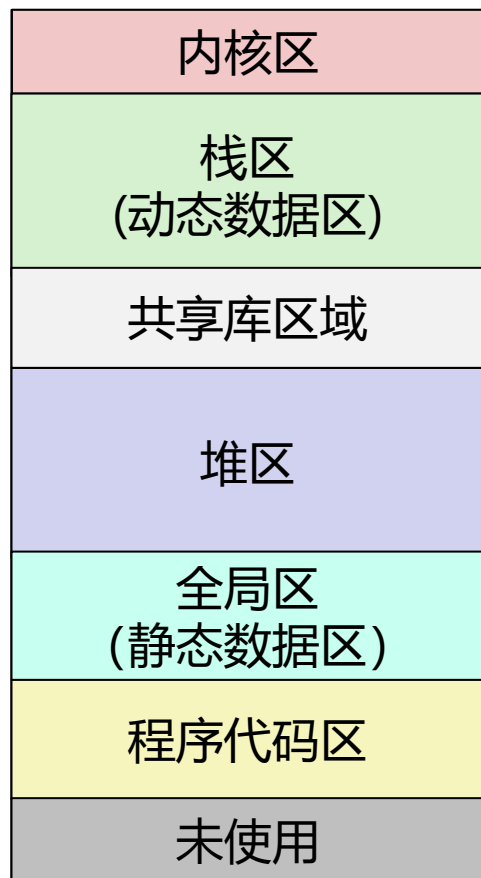
- ◆ **全局变量和局部变量也可以同名**，此时，在局部变量的作用域内，同名的全局变量不可见。

```
int a=2, b=4; /* 全局变量a和b*/  
void func()  
{  
    int a; /* 局部变量a*/  
    a=3; /* 使用的是局部变量a*/  
    b=5; /* 使用的是全局变量b*/  
    printf("a=%d,b=%d\n", a, b); /*输出 a=3, b=5 */  
}  
int main(void)  
{  
    func();  
    printf("a=%d,b=%d\n", a, b); /*输出 a=2, b=5 */  
}
```

4、生存期

变量的**生存期**由变量的**存储方式**决定。

(1) 变量的存储方式： 指为变量分配内存存储空间的方式，分为**静态存储方式**和**动态存储方式**。



程序的线性地址空间

- ◆ **静态存储方式：** 是指在程序运行之前，系统就为变量在**全局数据区**分配存储单元，并一直保持，直到整个程序结束。
- ◆ **动态存储方式：** 是在程序运行过程中，当执行到变量所在的代码块时才由系统在**栈区**（**动态数据区**）为变量分配存储单元，而在退出程序块时立即释放变量的内存空间。

(2) 变量的生存期： 指一个变量在程序执行过程中的有效期，
即能在内存中存在多久。

变量的生存期由变量的存储方式决定。

- ◆ **采用静态存储方式分配的变量：** 在整个程序执行过程中始终占据固定的存储单元，其**生存期是整个程序的执行时间**——称为**长生存期**或**永久生存期**。
- ◆ **采用动态存储方式分配的变量，** 在程序执行过程中动态地进行分配和释放：**进入代码块时因分配空间而存在，退出代码块时因释放空间而消亡**，它们的生存期是其所在代码块的执行期间——称为**短生存期**。

5、存储类型

- ◆ **存储类型**是变量和函数的一个基本属性。
 - 决定着变量的**作用域**、**存储分配方式**、**生命周期**和**初始化方式**。
 - 决定着函数的**作用域**。

◆ 存储类型的声明

在定义变量或函数的时候，在类型名前面加**存储类型修饰符**
(存储类型关键字) 以声明相应对象的存储类型：

存储类型 类型名 变量名；

- ◆ 变量的存储类型关键字有：**auto**、**extern**、**static**、**register**
- ◆ 函数的存储类型关键字有：**extern**和**static** (5.9节)

(1) 存储类型**auto**：用于声明自动变量

- ◆ **局部变量**的缺省存储类型是**auto**，也称为**自动变量**。

如：**auto int a;** /*一般省略**auto**：**int a;**

也可以省略为：**auto a;** */

在函数体或程序块内定义的普通局部变量都是自动变量

- ◆ 特点：

- **块作用域**：变量的作用域仅在所在的函数或程序块内。
- **动态分配方式**：进入程序块时在**栈区**分配内存，退出块时空间被释放。
- **短生存期**：变量的生存期只存在于其所在块的执行期间。
- **初始化**：定义时若没有显式初始化，其初值是不确定的；
若有显式初始化，则每次进入块时都要执行一次赋初值操作。

自动变量的产生、使用和消亡：

- ◆ 在程序执行到自动变量所在的程序块（函数体或复合语句）之前自动变量实际是不存在的 —— 系统尚未给它们分配内存存储空间。
- ◆ 当程序执行到自动变量所在的程序块那一刻，系统为程序块中的自动变量分配内存空间 —— 从此刻开始自动变量开始存在。
- ◆ 在程序块的执行期间，其中的自动变量一直存在。
- ◆ 在程序块执行完的那一刻，系统会自动释放（回收）其中自动变量的存储空间 —— 自此程序块中的自动变量就不复存在，不能再使用。

—— **自动变量的生存期只在程序块的执行期间。**

之后，如果程序块再次被执行（如再次调用了函数或循环中重复执行一个程序块），则在再次进入程序块的那一刻，再次创建自动变量、再次使用、再次释放，如此重复。

```
int main()
```

```
{
```

i在main的函数体内有效

```
    int i;
```

此刻创建i

```
    i=...;
```

```
    for(i=0; i<10; i++)
```

k仅在循环体内有效

```
    {
```

```
        int k;
```

每次进入循环体时在此创建k

```
        k=i*i;
```

```
        printf("%d ", k);
```

```
    }
```

每执行完一次循环体时在此销毁k

```
    ...
```

```
}
```

此刻销毁i

(2) 存储类型extern：声明外部变量

- ◆ **全局变量**隐含的存储类型是**extern**，也称为**外部变量**。

但在定义时不要使用关键字extern。

如：**int magic;**

定义外部变量时不能写为
extern int magic;



- ◆ **特点：**

- **文件作用域：**全局变量的作用域是从定义之处开始，直到文件结束。
(还可通过**extern声明**，将外部变量的作用域扩大到其他位置)
- **静态分配方式：**在整个程序开始执行的那一刻系统为外部变量在**静态**
区分配存储单元，直到整个程序运行结束后才由系统收回。
- **永久生存期：**全局变量的生存期存在于整个程序的执行期间。
- **初始化：**定义时若没有显式初始化，则其初值自动置为0，如 **int a;**
若有显式初始化，则在创建时执行一次赋初值操作，如 **int a=2;**

◆ 函数的存储类型——外部函数

函数默认的存储类型都是**extern**的，也被称为**外部函数**。

因此**函数都是全局的**，其作用域属于文件范围，对程序的任何部分都可见——即可为**（至少本文件中的）其他函数调用**。

在**函数定义**时可以使用关键字**extern**，但一般都省略。

如：

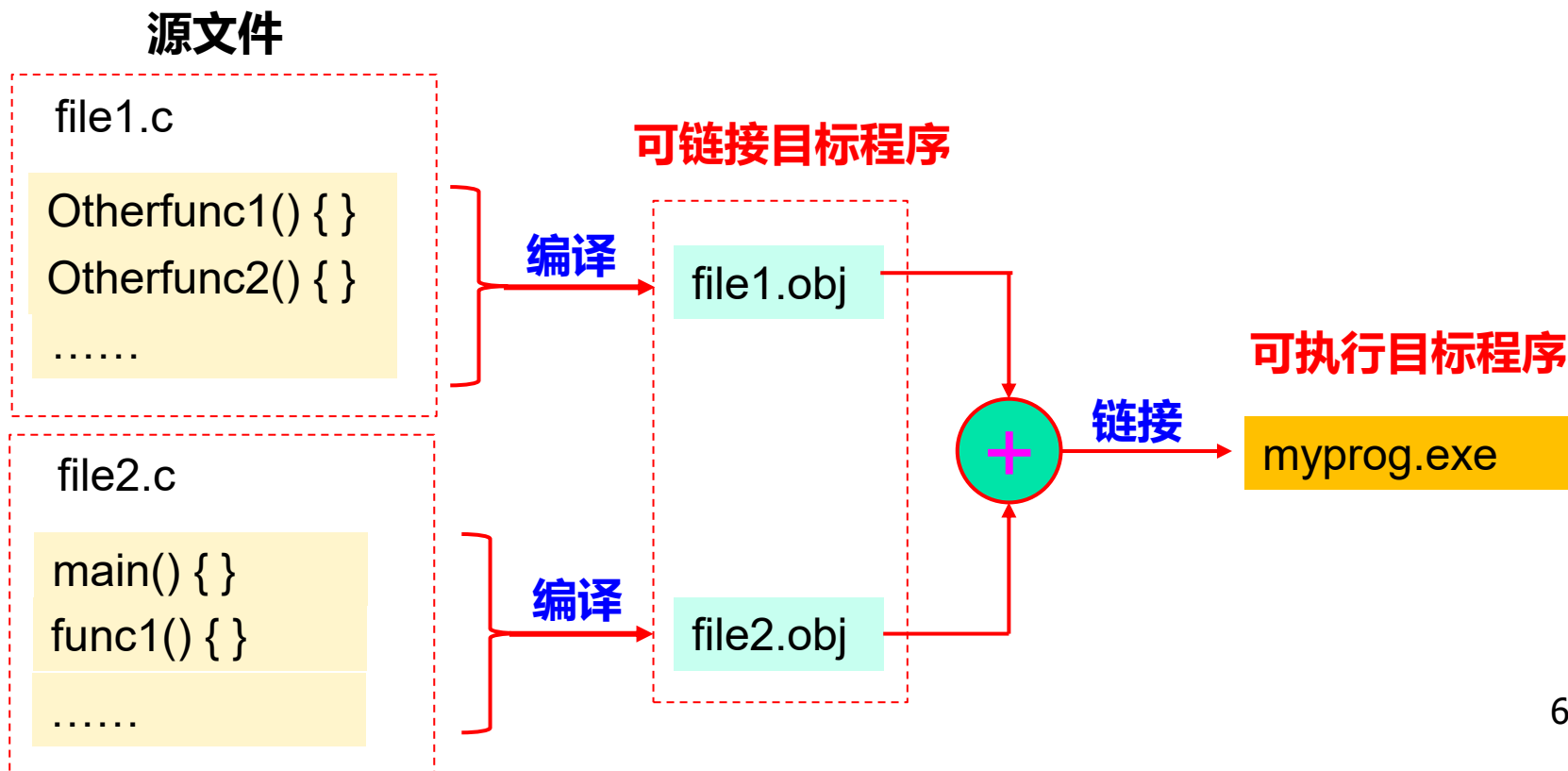
```
extern double func(int x)
{
    ... /*函数体*/
}
```

或：

```
double func(int x)
{
    ... /*函数体*/
}
```

C程序的多文件结构和外部声明

- 对于“迷你”程序，可以把所有的代码写在一个.c文件中。
- 但通常对于大一点的程序，通常是把程序里的函数写在几个.c文件中，每个.c文件单独编译，最后统一链接成一个完整的可执行文件。



- ◆ 每个.c文件中都可以定义一些全局变量和函数，这些符号（全局变量名、函数名）的初始作用域只在本文件中。
- ◆ 如何在文件中一个文件中使用另一个文件中定义的变量或函数呢？

使用**extern声明**（外部声明）扩展符号的作用域：

- ◆ 外部变量声明的方式：**extern 类型名 全局变量名；**

如：**extern int a;**

- ◆ 外部函数声明的方式：**extern 函数原型；**

如：**extern double func(int x);**

file1.c

```
int a=3;
extern int b;
extern void func1(int x, int y);
void otherfunc1( )
{
    .....
    b=2*a;
    .....
}
int otherfunc2(int x)
{
    int y;
    .....
    func1(x, y);
    .....
}
```

file2.c

```
int b=10;
extern int a;
extern void otherfunc1( );
extern int otherfunc2(int x);
int main()
{
    .....
    otherfunc2();
    .....
}
void func1(int x, int y)
{
    .....
    otherfunc1();
    .....
}
```


注意事项:

- ◆ **对外部变量作外部声明时不要有初始化**: `extern int a=3;` ✗
不要漏写extern。 **正确的外部变量声明**: `extern int a;` ✓
- ◆ 函数定义和作函数外部声明时都可以加extern。
- ◆ 只能对外部变量作外部声明，局部变量没有外部声明的用法。

一般把变量定义语句称为**变量的定义性声明**；而把全局变量的外部声明语句称为**变量的引用型声明**；外部(全局)变量有变量定义和变量引用两种声明形式；局部变量只有定义声明，没有“引用声明”。

- ◆ **对静态全局变量不能作外部声明**。

外部变量和外部函数的**定义性声明**只能有一处，但**引用性声明**可以有任意多处、可以写在任意位置。

➤ **在文件作用域内的外部声明**，其“声明的作用域”在文件范围内，一次外部声明后，后面其它位置可以任意使用。

➤ **在块作用域内的外部声明**，其“声明的作用域”只在本块范围内，本块内可以使用，但块外的其它位置如果想用必须重新进行外部声明。

```
int a=3;      变量定义
extern func1(int x, int y);
otherfunc1()
{
    extern int b;      变量声明
    .....
    b=2*a;
    func1(a, b);
    .....
}
otherfunc2(int x)
{
    extern int b;      变量声明
    .....
    func1(x, b);
    .....
}
```

(3) 存储类型**static**: 声明静态变量

在定义变量时使用**static**声明的变量称为**静态变量**。

- ◆ 用**static**定义的**局部变量**，称为**静态局部变量**。
- ◆ 用**static**定义的**外部变量**，称为**静态外部变量**。

性质: 不管是静态局部变量还是静态外部变量，都是

- ◆ **静态存储分配方式**
- ◆ **长生存期**
- ◆ **缺省初值为0**；赋初值只在定义时执行一次初始化

它们的不同之处在于：作用域不同。

1) 静态局部变量

- ◆ **块作用域**：和自动变量一样，只在定义它的块中可用。
- ◆ **长生存期**：静态局部变量也在**静态数据区**分配存储空间、而不是在栈区，其所在的程序块一次执行结束时，它的存储空间也不会被释放，而是一直保持到整个程序执行结束才会被系统收回。
- ◆ **记忆性质**：因其存储空间在退出程序块时不会消失，其中的值也就保存下来。下次再进入块时，不会重新为其分配空间，而是继续使用第一次分配的空间，且其值等于上次进入退出时保存下来的值，可以在本次执行中继续使用，这称为静态局部变量的**记忆特性**。
- ◆ **初始化**：在初次进入程序块执行时做一次赋初值操作。如果有显式初始化，则用指定的初值初始化；如果没有显式初始化，则自动初始化为0。其后再次进入程序块执行时不再进行初始化操作，而是直接使用上一次的值。

静态局部变量 Vs. 自动变量：

- ◆ 自动变量的存储空间在进入程序块时临时分配，在退出程序块时会自动释放，其值也就不存在了。再次进入程序块时，会重新分配存储空间、重新“初始化”——**新的存储空间和初值**，和上次进入没有任何关系，因此**自动变量没有记忆性质**。
- ◆ 静态局部变量在程序运行时一次分配好空间，其后只用这个空间，在程序结束前都不变。
- ◆ 静态局部变量虽具有静态内存、长生存期，但其作用域还是和自动变量一样，只是**块作用域**，只在定义它的块中有效，只能在块内使用，**超出块的区域不能引用**（如果用指针访问可以例外）。

例5.8 编程计算 $1!+2!+3!+4!+\dots n!$

分析：由于 $n!=n*(n-1)!$ ，所以求 $n!$ 可以将 n 乘到 $(n-1)!$ 直接获得，而不需要每次都从1乘到 n 。

定义函数 $\text{fac}(n)$ ，利用静态局部变量的记忆特性实现 $n!$ 的计算：

```
long fac(int n)
{
    static long f = 1;    /* 静态局部变量，初始化为1 */
    f *= n;                /* 将n直接乘到上次的 f 值上（即(n-1)!的值）从而
                           可得本次的n!的值*/
    return f;              /* 每次返回时，f 中保留有本次计算的n!值*/
}
```

```
long fac(int n)
{
    static long f = 1; /* 静态局部变量 */
    f *= n;
    return f;
}
```

```
int main(void)
{
    int i, n;
    long sum = 0;
    printf("input n(n>0):\n");
    scanf("%d", &n);
    for (i=1; i<=n; i++)
        sum += fac(i);
    printf("1!+2!+..+%d!=%ld\n", n, sum);
    return 0;
}
```

执行过程分析：
设输入n的值等于5。

main中 for循环 的i值	第i次执 行fac 函数， 进入时 f的值	第i次 执行 fac函 数，退 出时f 的值	第i次 执行 fac函 数的返 回值	main 中sum 在每 次循 环后 的值
1	1	1	1	1
2	1	2	2	3
3	2	6	6	9
4	6	24	24	33
5	24	120	120	153

2) 静态外部变量

- 静态外部变量和普通的外部变量的唯一区别是**作用域**：**静态外部变量的作用域只在定义它的文件内，在其它文件中不能使用**。而普通外部变量的作用域用**extern**声明后可以扩展到其它文件。
- 不要用**extern**对静态外部变量进行外部声明。
- 其它性质，如**静态存储**、**永久生存期**、**初始化**等，和普通外部变量完全一样。

```
/* file1.c */
#include <stdio.h>

static int a;
int b;

int main( )
{
    .....
}

int myotherfunc()
{
    .....
}
```


为什么会有静态外部变量？

一个文件中定义的静态外部变量的作用域只在本文件中，这使得**可以在不同的文件中定义同名的静态全局变量，它们相互独立，互不影响。**

- ◆ **好处：**当多人分别编写一个大程序的不同文件时，可以按照各自需要定义自己的静态全局变量而不用考虑是否会与其它文件中的静态全局变量同名，**保证了文件的独立性。**
- ◆ **普通全局变量必须保证整个程序中不能有重名的其他全局变量，否则编译出错。**

例5.9 伪随机数发生器的实现与使用（课下练习）

产生随机序列需要给定初始种子，因此种子**seed**是各随机数发生器函数所共享的变量，应定义在函数外。而且，seed只提供给srandom、random和probability等产生随机数的函数操作，并不希望任何其他函数访问它们，因此将函数srandom、random和probability，以及它们所操作的种子seed设计在一个源文件file1.c中，并在定义seed时加上static，限制其作用域只在本文件内。将函数main设计在另一个源文件file2.c中。

由于**seed是静态外部变量**，它对本文件的这些函数来说是共享的，它的值在函数调用之间被保存下来了。但是，它对本文件来说是私有的，在文件file2.c中不能用extern对seed做声明，也不能在main函数中使用seed。这样可以在文件file2.c中调用这些随机数发生器函数而不必担心副作用。

源程序\file1.c

```
#include <time.h>
#define INITIAL_SEED      17
#define MULTIPLIER        25173
#define INCREMENT         13849
#define MODULUS           65535

static unsigned long seed = INITIAL_SEED; /* 随机数的种子 */
void srand( )
{
    seed = time(NULL); /* 用系统时间初始化随机数的种子 */
}
unsigned random(void)
{
    seed = (MULTIPLIER * seed + INCREMENT) % (MODULUS + 1);

    return seed;
}
double probability(void)
{
    seed = (MULTIPLIER * seed + INCREMENT) % (MODULUS + 1);

    return (seed / (double)( MODULUS));
}
```

```
#include <stdio.h>

extern void srandom(void);      /* 函数原型 */
extern unsigned random(void);  /* 函数原型 */
extern double probability(void); /* 函数原型 */

int main(void) /* seed在main不可用 */
{
    int i;
    srandom();
    for (i= 0; i<6; i++)
    {
        printf("%u\t", random());
        printf ("%f\n", probability());
    }
    return 0;
}
```

3) 静态函数

在函数定义时使用关键字**static**修饰的函数称为**静态函数**。

如： **static** double power(int x,int n); /* 静态函数 */
 {

 }

特性： **静态函数的作用域只在定义它的文件中**，即只能在本文件中使用，其他文件不能使用，故不能做extern声明来扩展它的作用域。

好处： 保证文件的独立性， **不同的文件可以有同名的静态函数**而互不影响。

(4) 寄存器变量

定义时用**register**声明的**局部变量**称为**寄存器变量**：

如： **register int i;** /* 等价于register i; */

for (i=0; i<=N; i++) { ... }

作用： register声明**建议**编译器把该变量存储在计算机的高速硬件寄存器中，除此之外，其余特性和自动变量相同，**但不能取地址**。

好处： 可以提高对这些变量的存取速度，从而可以提高程序的执行效率。
所以，一般可以把程序中访问最频繁的变量声明为register型变量。
但不可多用：**寄存器有限**，不一定能分到可用的寄存器，否则仅作普通变量处理。

另，能否用register要具体视编译器的策略而定，未必有效。**且只能对局部变量指定register。寄存器变量不能取地址。**

5.4 递归函数

如果一个函数的函数体中出现了**对自己的调用**，则称该函数为**递归函数**。对自身的调用行为称为**递归调用**。

1) 递归调用有两种形式

(1) **直接递归调用**：在自己的函数体中出现对自己的调用。

```
int f(int x)
{
    int y, z;
    .....
    z = f(y);
    .....
    return(2*z);
}
```

◆ 此处f调用了自己。

◆ y是递归调用时使用的实参。

(2) 间接递归调用：一个函数的执行过程中转了个弯又调用到了它自己。

如：

```
int g(int x)
{
    int y, z;
    .....
    z = h(y);
    .....
    return (z);
}
```

g函数中调用了h函数

```
int h(int n)
{
    int m;
    .....
    m = g(n);
    .....
    return m;
}
```

h函数中又调用了g函数

以下以直接递归为例

(2) 递归的本质——循环执行递归函数自己的函数体

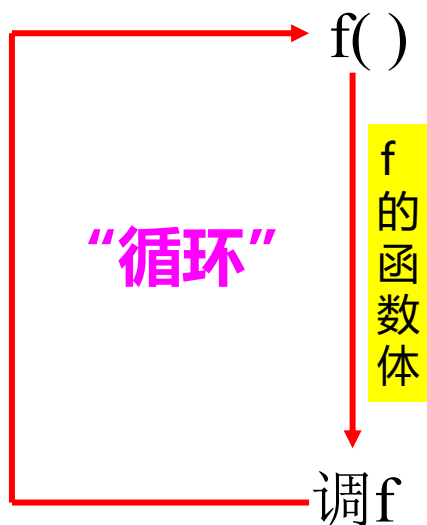
```
int f(int x)
{
    int y, z;
    .....
    z = f(y);
    .....
    return(2*z);
}
```

设想：如果f调用的是其它函数，其行为就是参数传递，然后转向被调用函数执行，再后返回。

那么，调用自己是一种什么行为呢？

◆ 理论上和调用其它函数是一样的：

递归函数也是一种函数，只是函数体中调用了自己。因此，发生递归调用时，也是仅在调用处用“实参”进行参数传递，然后以该实参再次执行一遍自己，如此循环。



以求n!为例进行说明

❖ **n!的数学公式**：

$$n! = \begin{cases} 1 & (n=0,1) \\ n*(n-1)! & (n>1) \end{cases}$$

如果令**fac(x)**是求整数x的x!的函数，则调用函数fac求n的阶乘的函数表示就是：**fac(n)**；

(1) 根据数学定义，**将n乘到(n-1)!上就可以得到n!**。

(2) **(n-1)!的阶乘怎么求呢？—— fac(n-1)。**

所以用fac函数表示n!的计算，可以表示为：

$$\text{fac}(n) = \begin{cases} 1 & (n=0,1) \\ n*\text{fac}(n-1) & (n>1) \end{cases}$$

将 $\text{fac}(n) = \begin{cases} 1 & (n=0,1) \\ n * \text{fac}(n-1) & (n>1) \end{cases}$

翻译成C函数，有以下描述：

```
long fac(int n)
```

```
{
```

```
    long f;
```

```
    if(n==0||n==1)
```

```
        f=1;
```

```
    else f=n*fac(n-1);
```

```
    return(f);
```

递归调用

```
}
```

```
#include <stdio.h>
```

```
void main( )
```

```
{
```

```
    int n;
```

```
    scanf("%d",&n);
```

```
    if(n<0) printf("error input!");
```

```
    else printf("%ld\ n", fac(n));
```

```
}
```

初次调用

以n=5为例说明**fac(5)**的执行过程：

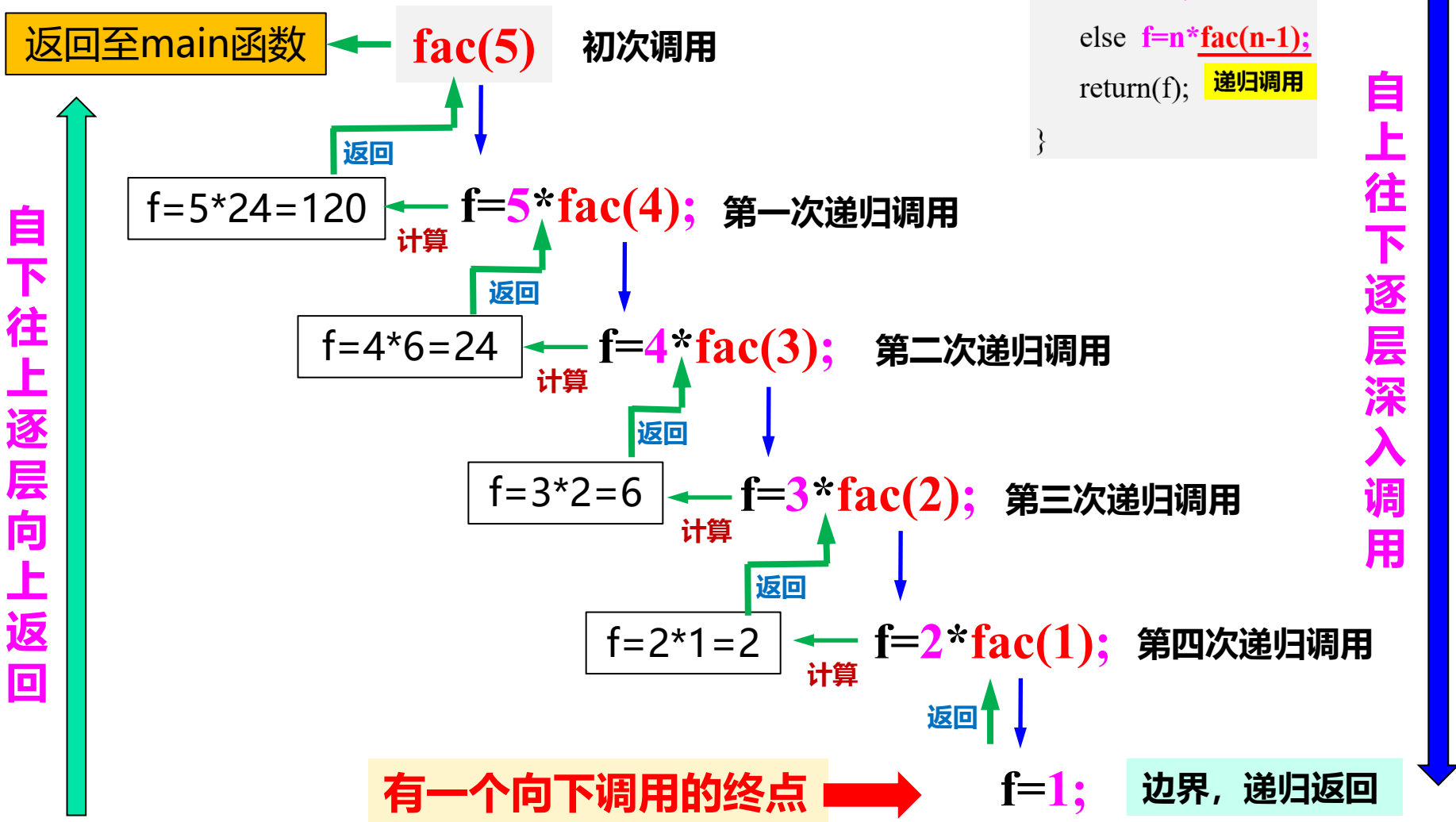
```
printf("%ld\n", fac(n));
```

```
long fac(int n)
```

```
{  
    long f;  
    if(n==0||n==1)  
        f=1;  
    else f=n*fac(n-1);  
    return(f); 递归调用  
}
```

自上往下逐层深入调用

自下往上逐层向上返回



可见：递归调用就是从初始调用开始，逐层向下**有限次**反复执行自己的过程：

① **每次递归调用都用新的实参重复执行一遍自己；**

② **经过有限次重复，最后必须结束递归调用，否则陷入无限循环；递归调用结束的条件称为递归终止条件（或递归边界条件）；**

③ 每次递归调用的实参会在本层参数的基础上有所改变（如减小 n ），而该改变将使得下一次的执行逼近递归结束条件；最后到达结束条件而终止继续向下的调用，然后开始逐层向上返回；

④ 从最下层开始返回，直到返回到初次调用的位置，递归函数的执行结束。



递归函数的设计要点：

- (1) 递归计算的规则：逐层向下做深层次调用，完成复杂计算；
- (2) 递归结束的条件：结束向下的递归调用，直接返回结果。

分治策略的设计思想：

对于复杂问题的求解：可以把大问题分解成为若干个小问题，然后通过求解小问题进而获得大问题的解。**如果分解出来的小问题性质和原始问题相同**，这样的求解策略就称为**分治策略**。

用分治策略求解问题的过程可以用递归描述：

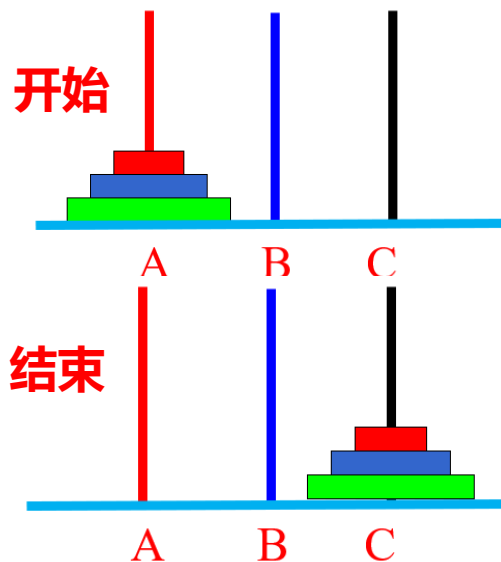
设 f 函数是对规模为 n 的问题进行求解的函数，则对规模较小的子问题，因其性质和规模为 n 的大问题一样，所以可以用 f 函数对规模较小的子问题进行**递归调用**求解。

注：子问题的分解是逐层向下进行的：原始问题规模最大，经过初次划分，可以分解出一些规模较大的子问题。如果这些子问题规模还比较大，不容易直接求解，则可以继续分解，直至分解出最简单的子问题，进而可以直接求解并递归返回。

汉诺塔问题

十九世纪末，欧洲珍奇商店出现一种叫**汉诺塔**游戏。推销材料介绍说：古代印度布拉玛神庙里的僧侣们正在玩这种游戏，如果游戏结束，**世界末日即来临**。

规则：有三根针**A、B、C**。A针上串有n（64）个大小不一的盘子，**大盘子在下、小盘子在上**。现将A针上的n个盘子移到C针上，**规则是：**每次只能移动一个盘子，且整个过程都要**小盘子在大盘子的上面**。



问如何实现？

事实：需移动的次数是 $2^n - 1$ 。

若有 $n=64$ 个盘子，则移动的总次数为：

$$2^{64} - 1 = 18,446,744,073,709,551,600$$

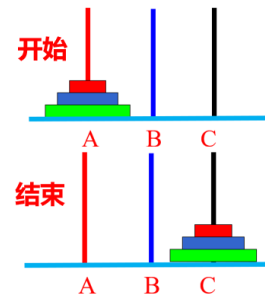
假定一秒钟可以移动一个盘子：

一年的秒数是： $365 \times 24 \times 60 \times 60 = 31536000$

移动64个盘子的时间是：

$$18446744073709511600 \div 31536000$$

= **58494217355**年，即：完成这个游戏需要**584.9亿年**



从能源角度推算, **太阳系寿命只有150亿年**，所以这个问题被称为**世界末日问题**。

设计思路

原始问题： n 个盘子在A针上，B针空，要移到C针上

即：借助B针，将A针上的 n 个盘子移到C针上。

原始问题可以分解为以下三个子问题来求解：

◆ **子问题1：** $n-1$ 个盘子在A针上，C针空，移到B针上；

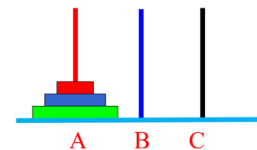
即：借助C针，将A针上的前 $n-1$ 个盘子移到B针上。

◆ **子问题2：** 子问题1完成后，将A针上剩下的最大盘子移到C针上；
直接移动即可（最简单子问题，不用进一步分解）

◆ **子问题3：** $n-1$ 个盘子在B针上，A针空，要移到C针上

即：借助A针，将B针上的 $n-1$ 个盘子移到C针上。

求解过程的形式化描述:



令函数 $f(n, x, y, z)$ 表示将放在x针上的n个盘子, 借助y针移到z针上的操作。

- ◆ **原始问题描述:** $f(n, A, B, C)$, 即将放在A针上的n个盘子, 借助B针, 移到C针上;
- ◆ **子问题1描述:** $f(n-1, A, C, B)$, 即将放在A针上的前n-1个盘子, 借助C针, 移到B针上;
- ◆ **子问题2描述:** $move(A, C)$, 即将A针上的最后一个盘子移到C针上;
- ◆ **子问题3描述:** $f(n-1, B, A, C)$, 即将放在B针上的n-1个盘子, 借助A针, 移到C针上

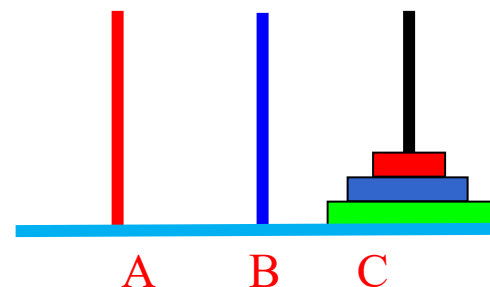
完成了上面的三个子问题, 原始问题也就解决了



依据上述思想，可设计**求解汉诺塔问题的函数** $f(n, x, y, z)$ 。
上述算法中每一步对 f 的调用就是一次**递归调用**。递归调用的结束条件是 $n=1$ 。

程序描述如下：

注：将 f 函数名命名为**hanoi**函数。



例 用递归方法求解Hanoi（汉诺）塔问题的程序

```
#include <stdio.h>
```

```
void move(char x, char y)
```

```
{
```

```
    printf("%c--->%c\n",x, y);
```

```
}
```

```
void hanoi(int n, char one, char two, char three)
```

```
{
```

```
    if(n==1) move(one, three);
```

```
    else {
```

```
        hanoi(n-1, one, three, two);
```

```
        move(one,three);
```

```
        hanoi(n-1, two, one, three);
```

```
    }
```

```
}
```

```
void main()
```

```
{
```

```
    int m=3;
```

```
    hanoi(m,'A', 'B', 'C');
```

```
}
```

只有一个盘子时

递归1

递归2

D:\CTest\test.exe

```
A--->C  
A--->B  
C--->B  
A--->C  
B--->A  
B--->C  
A--->C
```

5.7 C11新增属性（自学）

1、函数修饰符 `_Noreturn`

- `_Noreturn` 是C11增加的关键字，用在函数定义的前面，告诉编译器这个函数**不会返回到调用处**，其结果是让编译器知道调用说明为`_Noreturn`的函数之后的代码不可到达。

```
如： _Noreturn void f ()      /* f函数没有返回，所以说明为_Noreturn */  
    {  
        abort();              /* 终止程序 */  
    }  
  
_Noreturn void g (int i)    /* 假如 i <= 0，将导致未定义的行为 */  
    {  
        if (i > 0) abort();  
    }
```

2、存储类型 `_Thread_local`

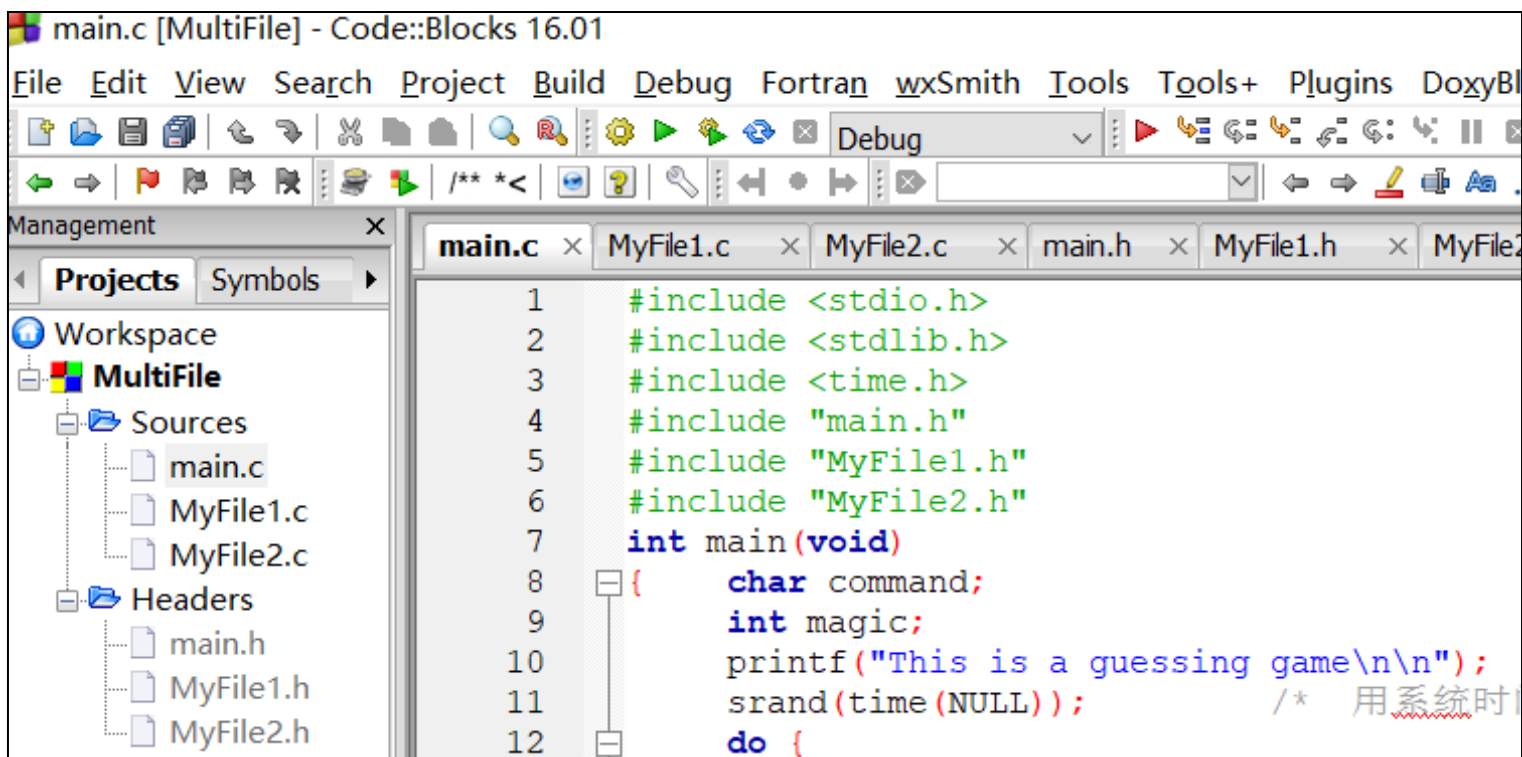
C11增加了`_Thread_local`存储类关键字以**支持多线程**。

进程和线程：都是操作系统的概念。进程是应用程序的执行实例，而线程是进程内部的一个执行单元。

- **系统创建好进程后**，实际上就启动了该进程的**主执行线程**。主执行线程以函数(比如main) 地址形式，将程序的启动点提供给Windows系统。主执行线程终止了，进程也就随之终止。
- 每一个进程至少有一个主执行线程，它无需由用户去主动创建，是由系统自动创建的。用户根据需要在应用程序中创建其他线程，多个线程并发地运行于同一个进程中。一个进程中的所有线程都在该进程的虚拟地址空间中，共同使用这些虚拟地址空间、全局变量和系统资源，所以线程间的通信非常方便。

再论C程序的结构（工程管理）

- 可以将一个C程序的多个函数写在**多个源文件**中，每一个C源文件中含有一个或多个函数的定义。
- ◆ IDE提供“工程”（Project）的手段管理这些源文件。



文件内容的组织:

- ◆ **.c文件**: 根据需要编写一个或多个.c文件, 每一个.c文件包含有一个或多个**函数的定义** (包括main函数所在文件)、本文件**外部变量的定义**等**定义性内容**;
- ◆ **.h文件**: 为每个.c文件编写一个对应的.h文件(一般与.c文件同名), 包含相关**数据类型声明** (枚举类型声明、结构类型声明、typedef类型声明等)、**外部变量声明** (extern声明)、**函数原型声明** (外部函数用extern声明)、**编译预处理指令**等**声明性内容**, 然后在每个使用到这些内容的.c文件中包含这些**头文件** (包括自己的.c文件)。

main.c [MultiFile] - Code::Blocks 16.01

File Edit View Search Project Build Debug Fortran wxSmith Tools Tools+ Plugins DoxyBl

Management

Workspace

MultiFile

Sources

main.c

MyFile1.c

MyFile2.c

Headers

main.h

MyFile1.h

MyFile2.h

main.c

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <time.h>
4 #include "main.h"
5 #include "MyFile1.h"
6 #include "MyFile2.h"
7 int main(void)
8 {
9     char command;
10    int magic;
11    printf("This is a guessing game\n\n");
12    srand(time(NULL)); /* 用系统时
```

main.c源程序

MyFile1.c [MultiFile] - Code::Blocks 16.01

File Edit View Search Project Build Debug Fortran wxSmith Tools Tools+ Plugins DoxyBlocks Settings Help

Management

Workspace

MultiFile

Sources

main.c

MyFile1.c

MyFile2.c

Headers

main.h

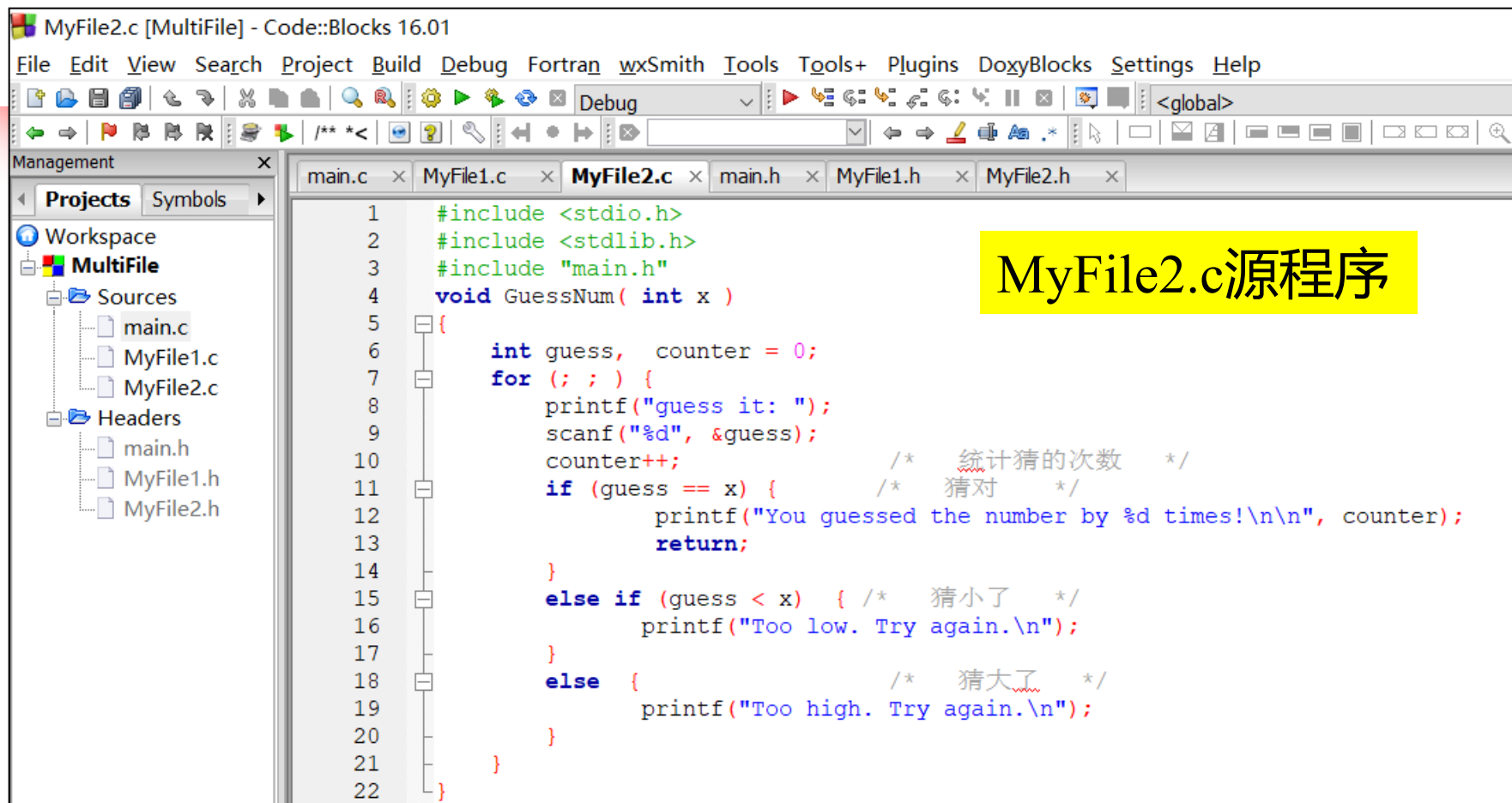
MyFile1.h

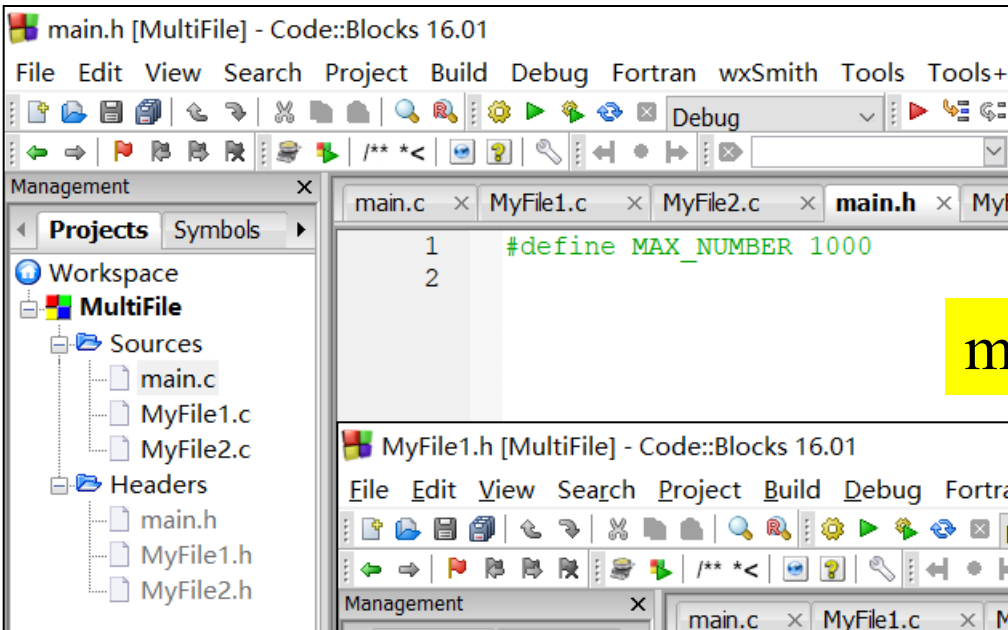
MyFile2.h

MyFile1.c

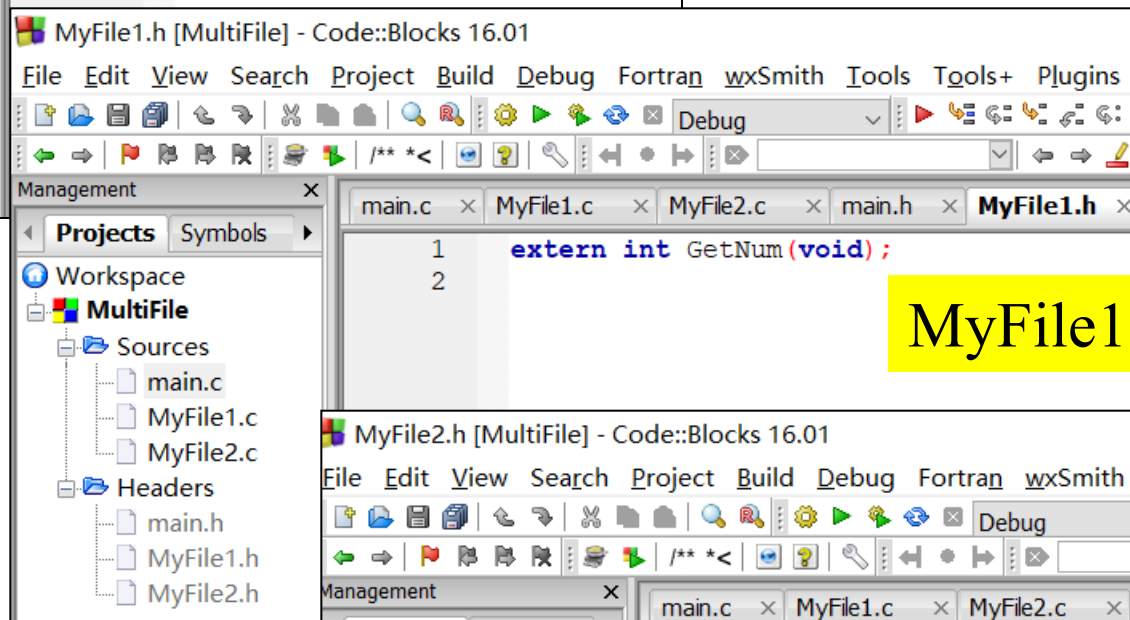
```
1 /******
2 函数名称: GetNum
3 函数功能: 产生一个1到MAX_NUMBER之间的随机数, 供游戏者猜测。
4 函数参数: 无
5 函数返回值: 返回产生的随机数
6 *****/
7 #include <stdio.h>
8 #include <stdlib.h>
9 #include "main.h"
10 int GetNum(void)
11 {
12     int x;
13     printf("A magic number between 1 and %d has been chosen.\n", MAX_NUMBER);
14     x=rand(); /* 调用标准库函数rand产生一个随机数 */
15     x= x % MAX_NUMBER + 1; /* 将这个随机数限制在1~MAX_NUMBER之间 */
16     return(x); /* 返回这个随机数给调用者, */
17 }
```

MyFile1.c源程序

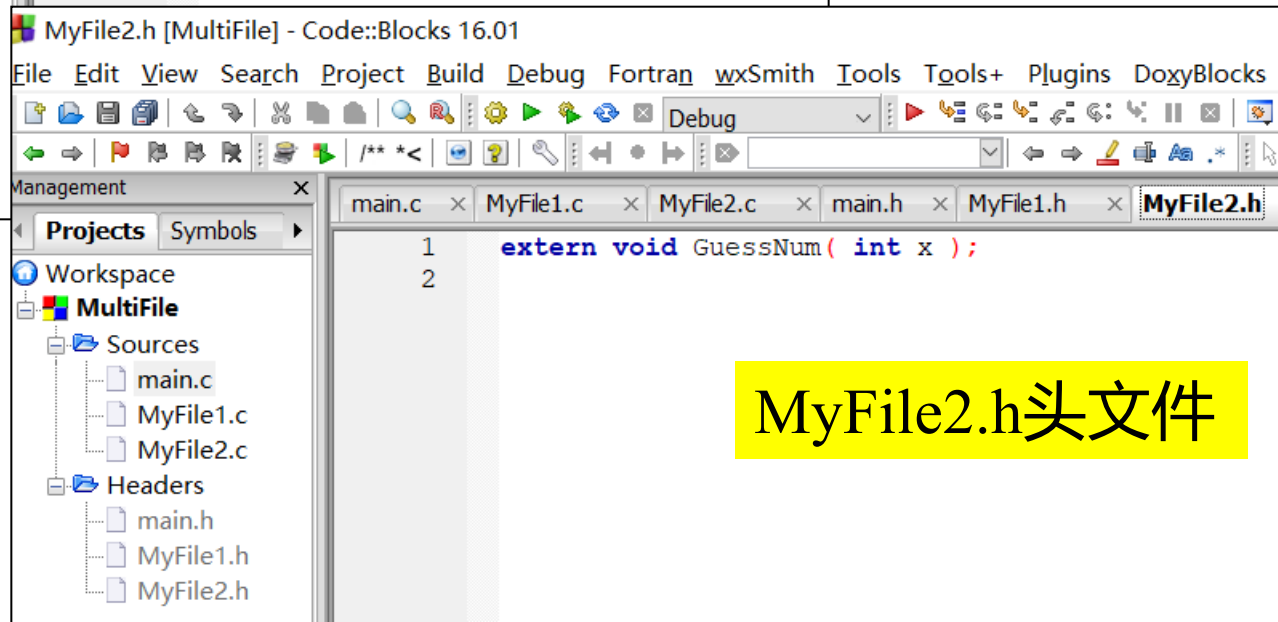




main.h头文件

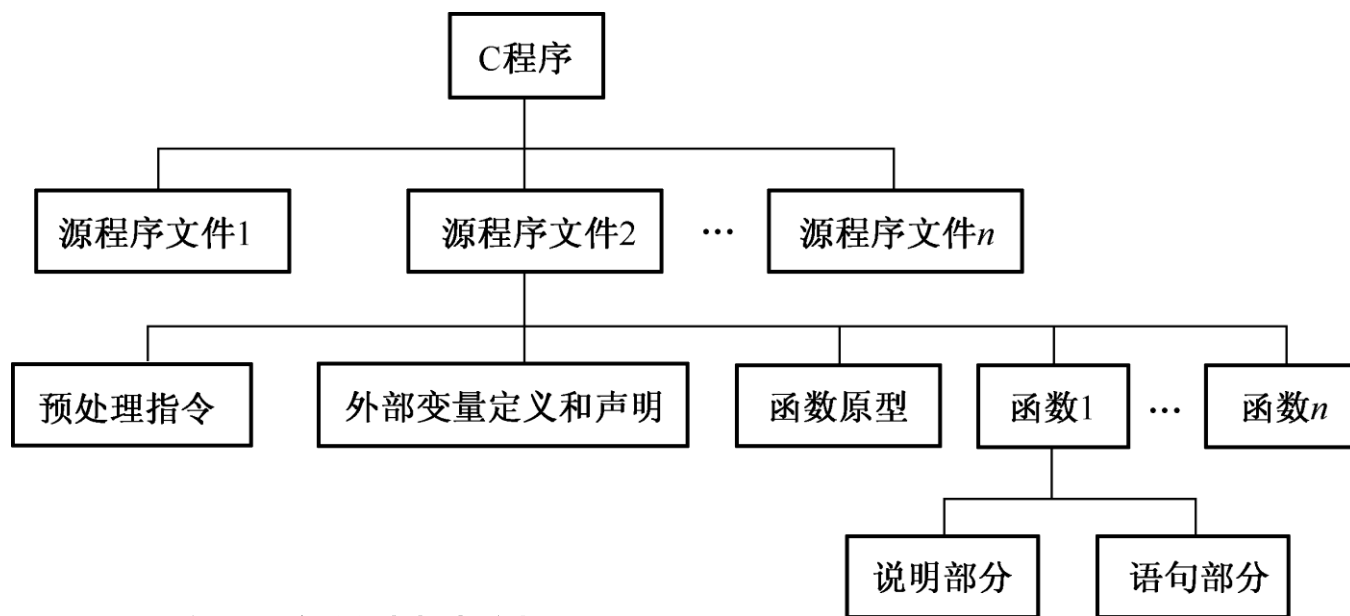


MyFile1.h头文件



MyFile2.h头文件

- 每个源文件（.c文件及其.h文件）可单独编译生成一个**可链接目标文件（.obj文件）**；
- 一个C程序的所有源文件都被编译之后，由**链接程序**将所有**可链接目标文件**（包括系统的标准库函数）装配成一个完整的**可执行目标文件，即可执行程序**。



C语言程序的基本结构

本章小结

- ◆ 结构化程序设计的基本思想
- ◆ 用C语言设计程序，任务只有一项，就是**编写函数**。
- ◆ 函数的定义、函数原型和函数的声明
- ◆ 函数的调用：参数传递、函数执行、函数返回。
 - C语言函数参数传递方式：值传递。
 - 地址传递的基本概念。
- ◆ 变量的作用域、可见性和生存期
 - 全局变量、局部变量的概念、定义及性质
 - 变量的存储类型：auto、extern、static、register
 - 函数的存储类型：extern、static